A DISCUSSIONS

The actor model also offers asynchronous communication and local state encapsulation. However, concurrency in actor systems is achieved through the dynamic creation of actors and pipelining [2], explicitly expressed via application logic. Due to the single-thread execution mode, developers must partition their applications into fine-grained actors. Determining the partitioning scheme, actor placement strategy across nodes, and the number of active actor instances imposes additional application design complexities, often involving performance and data consistency trade-offs, an unsolved research problem [88]. Last but not least, parallelizing a component on multiple actors would also make querying the component state consistently more complicated. State-of-the-art actor systems do not offer querying features across multiple actors. FaaS runtimes follow similar principles, so similar reasoning applies to them. In contrast, MODB does not require explicit state partitioning of a component during application development. Concurrency in MODB is achieved by partitioning annotations, obviating the cost and complexity of creating and placing new actor instances, and allowing developers to use SQL to query the entire state of a component.

Availability. Based on the CAP theorem, MODB is not an available system for transactions involving unavailable services, but, as stated in § 5.3, it can mitigate the impact of such unavailability. For transactions that involve only non-impacted DoMSes, the MODB system remains available. If dispatchers are unavailable, multi-DoMS transactions cannot be scheduled, but all read-only single-DoMS transactions or those that do not conflict with uncommitted batches can be executed.

External Effects. Externally observable side effects are possible. We assume the external entity is managed by an external system, outside the control of MODB and the external side effect can be compensated, in a similar spirit to Sagas [33]. During a transaction abort, MODB can trigger the compensation action in the external system, and any side effects will be compensated.

MODB does not bring a new programming language. Instead, its Java SDK provides a programming framework for Java and parses the added program constructs illustrated in (§ 3.2). §A.2 explains the details. In our benchmark implementations, VMS functions use Java syntax and resemble programs implemented in popular Java web frameworks, such as Spring and Quarkus, so migrating from these frameworks will involve minimal, if any, friction. Note that this was a design goal of our specification (§ 3.2).

MODB Java SDK is fully compatible with JVM testing frameworks like JUnit. SysX test units allow a developer to test a specific function in the invocation chain and reason about it. Please refer to the tests in our source code for details. For tracing, industry tools and APIs like Prometheus and OpenTelemetry can be easily integrated as any microservice would in practice.

Disaggregation. The VMS programming model does not forbid the separate scaling of computing and storage. The current implementation of the VMS database **adopts the in-process design to achieve high performance**, inspired by the success of SQLite and DuckDB. As discussed in § 6.1, it is possible to have separate scaling of computing/storage. In this direction, sharding the VMS state can be provided out of the box by a cloud-scale DBMS like Aurora, transparently to VMS function execution.

B DOMS BOOTSTRAP

Before initializing a DoMS instance, the SDK builds a logical representation (referred as `DoMSMeta`) of the DoMS that can be managed by MODB at runtime based on the DoMS specification provided by the user.

`DoMSMeta` is a set of objects that stores metadata about the data model, data constraints, and the mappings of events to application functions. These are leveraged to check the correctness of intra-DoMS properties, exploring the specification space to identify anomalies related to functions that append to its own input event log, event logs with multiple functions appending into it, functions without input event log, output events without associated event log, writes to foreign data items, writes to and reads from non-declared tables, queries that include non-existing fields, functions with more than an input event log, missing data dependence (i.e., incorrect foreign key declaration), and more.

These anomalies are detected by navigating through the abstract syntax tree (AST) of user-defined classes and correlating information extracted from the meta-programming constructs, e.g., annotations. If an anomaly is detected, the DoMS initialization procedure is stopped and the user is informed about the model violations.

Afterwards, MODB SDK partners with JVM bootstrap class loader to preload and inject system functionality into key user classes in the execution path of DoMS functions so to make their lifecycle and MODB internals transparent to developers. For instance, it is not a goal to expose how an output event is appended to an event log nor to disclose internal index structures used to store application state.

Specifically, concrete implementations of `@Repository` interfaces are created via bytecode generation and the generated code is instrumented to intercept database operations and forward them to an internal transaction management layer (§ 6). `@Repository` custom instances are then injected into preloaded `@Microservice` instances. Besides, complex queries (e.g., with aggregate, join, and where clauses) declared as part of `@Repository` interfaces are parsed and have their query execution plans generated in advance to speed up DoMS execution.

Apart from managing application entities that map to database rows, necessary in every ORM-based application, no other objects require being managed by users. The MODB SDK hides system complexities entirely, thereby allowing for clean application logic specification.

Besides intra-DoMS properties, inter-DoMS properties also need to be checked for correctness. In particular, a single DoMS instance appender per event log must hold, event schemas must match across DoMS instances, foreign tables from every DoMS must map to a single native table from another DoMS unique DoMS name identifiers, correctness of transaction definitions, and others.

After initialization, each DoMS instance forwards its `DoMSMeta` to the Catalogs, which are then combined for correctness check of inter-DoMS properties by the dispatchers. If some property is violated, the DoMS instances involved are not allowed to participate in multi-DoMS transactions. Afterward, the MODB bootstrap process is considered completed.

C ALGORITHMS

C.1 Extended Scheduling Algorithm

Based on Algorithm 2, we divide transaction scheduling in a DoMS into three functions, *Ingest*, *Schedule*, and *TxnCompletionCallback*, which are executed by different threads concurrently.

The *Ingest* prepares transaction inputs for scheduling by reading all input events available in the inbound event stream and building transaction contexts for execution. A transaction context maps an input event to the respective application function of a DoMS. The *Schedule* tracks the execution of transactions continuously, making a best effort to schedule the largest number of concurrent transactions possible, respecting the global order. Like the *Ingest*, it blocks at specific points to release resources for transaction execution. The *TxnCompletionCallback* is executed by the thread assigned to execute a transaction. It updates the *lastTidExec* and the sets *partSched* (set of partitioned transactions scheduled) and *paraSched* (set of parallel transactions scheduled) atomically to allow for the scheduling thread to progress with further transaction scheduling. In the event of an error, the abort procedure is triggered, as covered in Algorithm 3.

Algorithm 2: DoMS transaction scheduling

Input: Infinite input event stream $\mathcal{I} = (e_1, e_2, \ldots)$
Input: *lastTidCommitted* (0 by default)
Output: Infinite output event stream $O = (o_1, o_2, \ldots)$

1 $txns \leftarrow \emptyset$;// transaction context set
2 $lastTidExec \leftarrow lastTidCommitted$;
3 $paraSched \leftarrow \emptyset$;// parallel txns scheduled
4 $partSched \leftarrow \emptyset$;// partitioned txns scheduled

5 **Function** Schedule():
6 $txn \leftarrow txn' \in txns$: txn′.prevTid = $lastTidExec$;
7 **if** $txn.mode = SINGLE$ **then**
8 **if** $\neg(partSched \neq \emptyset \vee paraSched \neq \emptyset) \vee (\exists t : t \in partSched \cup paraSched \wedge t.type = RW)$ **then**
9 **block while** above cond. holds;
10 **if** *t.type=R* **then dispatch**(t); **else execute**(t);
11 **goto line 6**;
12 **do**
13 **if** $txn.mode = PARA \wedge (partSched = \emptyset \vee (\forall t : t \in partSched \wedge t.type = R))$ **then**
14 $paraSched \leftarrow paraSched \cup txn$;
15 **dispatch**(txn);
16 **else if** $txn.mode = PART \wedge (paraSched = \emptyset \vee (\forall t : t \in paraSched \wedge t.type = R))$ **then**
17 **if** $partSched[t.part] = \emptyset \vee (\forall t : t \in partSched[t.part] \wedge t.type = R)$ **then**
18 $partSched \leftarrow partSched \cup txn$;
19 **dispatch**(txn);
20 **else block until** $partSched[t.part] = \emptyset$;
21 **else**
22 **goto line 6**;
23 $nextTxn \leftarrow txn' \in txns$: txn′.prevTid = $txn.Tid$;
24 $execModeAux \leftarrow txn.mode$;
25 $txn \leftarrow nextTxn$;
26 **while** $txn \neq \emptyset \wedge execModeAux = txn.mode$;

27 **Function** Ingest():
28 **while** *true* **do**
29 $I' \leftarrow$ **pollAllBlocking**$(\mathcal{I})$ where $I' \subset I$;
30 **for** $e \in \mathcal{I}'$ **do**
31 $f \leftarrow$ **mapFunction**(e);
32 $txnCtx \leftarrow$ **buildTxnCtx**(e, f);
33 $txns \leftarrow txns \cup txCtx$

34 **Function** TxnCompletionCallback($txn, status$):
35 **if** $status = ERROR$ **then**
36 **abort**(txn);
37 **return**;
38 $lastTidExec \leftarrow max(lastTidExec, txn.Tid)$;
39 $txn.isFinished \leftarrow true$;
40 $O \leftarrow O \cup txn.result$;
41 **if** *txn.mode = PARA* **then**
42 $paraSched \leftarrow paraSched \setminus txn$;
43 **else if** *txn.mode = PART* **then**
44 $partSched \leftarrow partSched \setminus txn$;

C.2 Transaction Abort Algorithm

Algorithm 3 presents the abort procedure.

D SCHEDULING CORRECTNESS

In this section, we analyze the correctness of the DoMS scheduling algorithm. It is based on the assumption that users provide a correct specification of application function execution modes.

Theorem D.1. *Let $O = [T_1, T_2, \ldots, T_n]$ be a total order of transactions defined by the dispatchers, and S be the schedule produced by the DoMS. S is conflict-equivalent to O.*

Proof. The DoMS scheduling algorithm ensures that transactions with different execution modes {Single threaded, Partitioned, Parallel} are not scheduled concurrently. It puts transactions into a sequence of groups, where all the transactions in a group belong to exactly one of the three execution modes. Transactions in each group have TiDs that are greater than all the TiDs of the transactions in any preceding groups. Furthermore, transactions in each group are completely executed before all the transactions in the subsequent groups, except for read-only (R) transactions. The algorithm ensures that all the R transactions are dispatched after all the potentially conflicting RW transactions with smaller TiDs are executed. Therefore, the multi-version data engine can ensure the latter reads the version according to the schedule stated in O. In the following analysis, we only focus on RW transactions.

For each transaction group, there are three possible cases:

Case (i). All transactions are single-threaded transactions. The DoMS scheduling algorithm ensures that for any pair of single-threaded RW transactions, they are executed strictly according to the order in O without concurrency.

Case (ii). All transactions are partitioned transactions. Let $G(O)$ be the conflict graph of the transactions in this group induced by

Algorithm 3: DoMS transaction abort

Input: *txn* that led to a conflict or constraint violation
Input: Infinite input event stream $\mathcal{I} = (e_1, e_2, \ldots)$
Input: Transaction context set *txns*
Input: Set of DoMS tables T

```
block (I);// block Ingest polling I
block (txns);// block Schedule access to txns
if (txn abort is raised by this DoMS) then
    forward abort msg to COMMIT_HANDLER;
txnsToAbort ← ∀txn′ ∈ txns : txn′.Tid ≥ txn.Tid;
txns ← txns \ txnsToAbort;
for table ∈ T do
    for txnToAbort ∈ txnsToAbort do
        table.versions ←
         table.versions \ txnToAbort.writeSet;
if (DoMS is the first node in the transaction graph) then
    log txn abort event in DoMS state;
    find the successor and predecessor events of txn aborted
     in I;
    adjust dependencies;
    reinsert events after txn aborted in I;
else
    find the predecessor event of txn aborted in I;
    wait until successor event of the predecessor is sent
     through I;
lastTidExec ← predecessor.Tid;
unblock (I);
unblock (txns);
```

the total order O, and $G(S)$ be the corresponding conflict graph induced by the schedule S.

Let $T_i \rightarrow T_j \in G(O)$. This implies:

- T_i and T_j conflict.
- $T_i \prec T_j$ in O.

The algorithm ensures that RW transactions accessing conflicting partitions are never scheduled concurrently. Therefore, for any pair of conflicting transactions T_i and T_j such that $T_i \prec T_j$ in O, the conflicting operations from T_i precede those from T_j in S. Hence, the edge $T_i \rightarrow T_j$ exists in the conflict graph $G(S)$, preserving the conflict-equivalence with $G(O)$.

Case (iii). All the transactions are embarrassingly parallel transactions. As we assume that they have no conflicting access to the DoMS state, $G(O)$ of the transactions in this group does not have any edges. Therefore, the corresponding $G(S)$ does not violate any precedence relationship in $G(O)$.

All in all, the complete schedule S is conflict-equivalent to the schedule specified by O. □

E IMPLEMENTATION AND OPTIMIZATIONS

To provide a high-performance cross-stack DBMS that unifies state and event management, numerous optimizations are required, including network I/O, transaction scheduling, concurrent data access, query processing, and storage formats for both data and event logs.

Network I/O Processing. Network communication with the dispatcher and among DoMSes is achieved with TCP connections through asynchronous I/O [64]. This applies to both event logs and protocol messages. I/O threads are divided into two categories, receivers and senders. The OS kernel signals receivers about new network I/O events, which in turn unmarshal log entries from the buffers, deserialize individual input events, and queue them for transaction processing. Whenever output events are made available, senders dispatch them as log segments to the corresponding consumer DoMSes. Note that **there can be multiple consumers for an event**. If there is a failure to send (e.g., a DoMS crash or a network partition), log segments are placed in a pending buffer for later reattempt.

A configurable number of receivers are placed in a network thread pool and return when no network events are available. Senders run independently and are uniquely assigned a DoMS consumer to send log entries to. This prevents cache pollution, i.e., cache lines of network events evicting cache lines of the DoMS consumer, and vice versa. In this case, when output event logs are unavailable, senders yield their CPU time to other system threads.

Transaction Dispatching. Network I/O processing in dispatchers follows a design similar to a DoMS but is targeted at maximizing transaction request dispatch while not jeopardizing the TiD assignment process. To meet this goal, sender I/O threads never block. Instead, in the absence of new event inputs, they yield their CPU time to another thread. This minimizes waiting times for DoMSes because blocking would cause the I/O thread to park and unpark, a costly process for input generation. Receiver I/O threads function in the same way as in a DoMS. Dispatchers assign TiDs to multi-DoMS transactions and compute the dependencies of each DoMS involved. They operate as independent threads in an event loop mode (i.e., non-blocking), outside the context of a thread pool.

F HIGH AVAILABILITY

DoMS. The DoMSes can replicate their produced event logs to standby replicas for failure recovery. In this case, sender I/O threads, in addition to logging log segments, can stream them to replicas. The replicas process the log segments in the same way as the primary, but do not send output events to consumer DoMS. For failure detection, replicas rely on heartbeats received from primary nodes. Upon detection of a failure, replicas trigger a leader election. Similar to Raft [63], replicas start as candidates and rely on the number of votes they receive to determine whether their candidacy succeeds. In this case, it broadcasts the election result. As transactions execute optimistically (§ 5.1), a newly elected leader may not contain some log segments. Therefore, it is necessary to request possible missing log segments from upstream DoMS. Note that the Catalogs must also be checked for abort information of in-progress batches. In this case, replicas must proceed in the same way as the primary to safeguard correctness. In other words, Algo. 3 must also be executed. Thereafter, the system can return to normal operation.

Proxies. Proxies act as stateless load balancers. In case of failures or newly deployed proxies, they obtain from the Catalogs the latest DoMS metadata and transaction definition versions stored. This procedure repeats if an invocation fails due to a deprecated DoMS metadata or transaction definition.

Table 3: DoMS execution modes in TPC-C

Component	New Order	Payment	Order Status
Warehouse	Keyed on warehouse and district IDs	Keyed on warehouse and district IDs	Read-only, thus parallel
Inventory	Runs keyed on supplier warehouse IDs in experiments but can be keyed on item IDs for increased concurrency	-	-
Order	Parallel	Parallel	Read-only, thus parallel

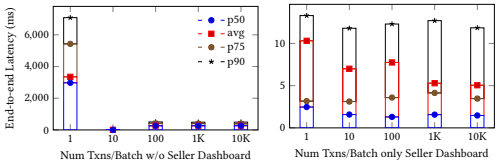

Figure 11: Effect of Batch Size on Latency

Table 2: DoMS execution modes in Online Marketplace

Component	Transaction	Primitive
Cart	Customer Checkout	Keyed on `customer_id`
Cart	Price Update	Single thread; can't extract from the event which carts contain the product
Stock	Customer Checkout	Runs single-threaded in the experiments but can be keyed on `seller_id`, `product_id`
Product, Cart, Stock	Product Delete	Keyed on `seller_id`, `product_id`
Product	Price Update	Keyed on `seller_id`, `product_id`
Order	Customer Checkout	Keyed on `customer_id`
Payment	Customer Checkout	Parallel
Shipment	Customer Checkout	Parallel
Seller	Seller Dashboard	Parallel

Dispatchers and Commit Handlers. Upon all *batch_complete* messages' arrival, a commit handler logs a *batch_commit* message to the Catalogs. Upon a crash, a stand-by replica requests the last committed batch ID from all DoMSes to cover cases where the commit handler crashes before logging it. Upon receiving the responses, it updates its last committed batch ID if necessary and sends a *batch_aborted* message to all DoMSes to abort all ongoing batches at the time of the crash. DoMSes then proceed similarly to a transaction abort. DoMSes discard non-committed data item versions, and TiD precedences are adjusted accordingly to receive transactions from the new process.

Differently from DoMSes the dispatchers do not stream event logs to replicas since these are simply transaction requests that can be retried by clients. Although this may lead to losing some useful work the DoMSes performed, it avoids a TCP round trip to each replica for every transaction emitted.

G DOMS EXECUTION MODES

Table 2 exhibits the DoMS execution modes used in Online Marketplace and Table 3 exhibits for TPC-C. In Dapr, all operations run as a parallel task given the lack of transaction isolation.

H POSTGRESQL TUNING

In the application ORM, we set the maximum number of connections to 10K, bypassing the default maximum of 100 connections, and we configure a connection pool to reuse connections. Besides, we disable synchronous commit, so updates are eventually logged rather than atomically[1] We use Unix domain socket for inter-process, kernel-based communication instead of the default TCP sockets. We increase the pgsql buffer size to improve analytical query performance. We increase the shared buffer from 128MB to 3GB, which corresponds to ~15% of 21 GB available in c5n.2xlarge instance, as suggested by PostgreSQL documentation. The work mem is set to 128 MB instead of the default 4MB.

[1]In off mode, there is no waiting, so there can be a delay between when success is reported to the client and when the transaction is guaranteed to be safe against a server crash (the maximum delay is 3X *wal_writer_delay*).

I ADDITIONAL EXPERIMENTS

I.1 Effect of Batch Size

To study the effects of the number of transactions per batch in OM, we start by fixing the batch epoch to infinity, varying the number of transactions in a batch, and maximizing the concurrency level. In Figure 10 (left), we observe an average of 1K tx/sec increase up to batch size 1K, the point where throughput stabilizes. Next, we fix the maximum number of transactions in a batch to infinity and vary the epoch. Figure 10 (right) demonstrates the effectiveness of the commit protocol, showing little variation across epochs.

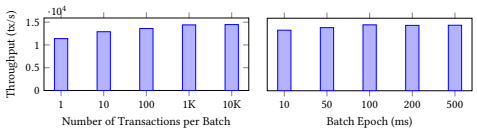

Figure 10: Effect of Batch Size on Throughput

Figure 11 (left) shows the substantial overhead of network I/O if using a single transaction per batch. It is worth noting that 10 transactions per batch achieves low latency and doubles Dapr's throughput, providing a good trade-off in this particular scenario. Besides, Figure 11 (right) shows the insensitivity of `Seller Dashboard`'s latency to the batch sizes. On the other hand, Figure 12 shows that varying the batch epoch leads to a more natural variation in latency for transactions committing in a batch, without affecting throughput though.

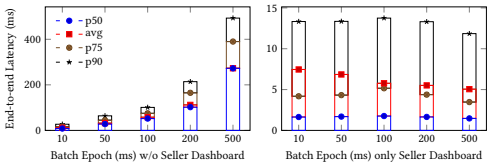

Figure 12: Effect of Batch Epoch on Latency

I.2 Effect of Scale Factor on Latency

Figure 13 reports the statistics of the intervals between the commit of each pair of adjacent batches for each scale factor in TPC-C. In MODB, batch latencies decrease because batches are filled and processed more quickly due to the higher number of requests. This effect is also observable in MODB with logging and checkpointing with an average of 17% overhead due to the intensive I/O incurred by the durable write procedures.

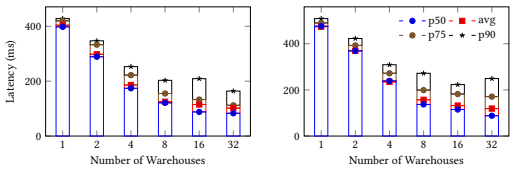

Figure 13: Effect of Scale Factor on Batch Latency in TPC-C. In-memory execution (left) and with durability (right)

J MODB TPC-C IMPLEMENTATION

```
@Inbound(values = "new-order-ware-in")
@Outbound("new-order-ware-out")
@Transactional(type = RW)
@PartitionBy(clazz = NewOrderWareIn.class, method = "getId")
public NewOrderWareOut processNewOrder(NewOrderWareIn in) {
    District district = this.districtDB.lookupByKey(new
          District.DistrictId(in.d_id, in.w_id));
    float w_tax = this.warehouseDB.getWarehouseTax(in.w_id);
    district.d_next_o_id++;
    this.districtDB.update(district);
    float c_discount = this.customerDB.getDiscount(in.c_id, in.d_id,
          in.w_id);
    return new NewOrderWareOut(in.w_id, in.d_id, in.c_id,
          in.itemsIds, in.supWares, in.qty, in.allLocal, w_tax,
          district.d_next_o_id, district.d_tax, c_discount);
}
@Inbound(values = "payment-in")
@Outbound("payment-out")
@Transactional(type = RW)
@PartitionBy(clazz = PaymentIn.class, method = "getId")
public PaymentOut processPayment(PaymentIn in) {
    District district = this.districtDB.lookupByKey(new
          District.DistrictId(in.d_id, in.w_id));
    district.d_ytd += in.amount;
    Warehouse warehouse = this.warehouseDB.lookupByKey(in.w_id);
    warehouse.w_ytd += in.amount;
    this.districtDB.update(district);
    this.warehouseDB.update(warehouse);
    Customer customer;
    if(in.by_name){
        List<Customer> customers =
              this.customerDB.getCustomerByLastName(in.c_d_id,
              in.c_w_id, in.c_last);
        int index = customers.size() / 2;
        if (customers.size() % 2 == 0) index -= 1;
        customer = customers.get(index);
    } else {
        customer = this.customerDB.lookupByKey(new
              Customer.CustomerId(in.c_id, in.d_id, in.w_id));
    }
    if (customer.c_credit.equals("BC")) {
        customer.c_data = "%d %d %d %d %d %f |
              %s".formatted(customer.c_id, in.c_d_id, in.c_w_id,
              in.d_id, in.w_id, in.amount, customer.c_data);
        if (customer.c_data.length() > 500) {
            customer.c_data = customer.c_data.substring(0, 500);
        }
    }
    customer.c_balance -= in.amount;
    customer.c_ytd_payment += in.amount;
    customer.c_payment_cnt += 1;
    this.customerDB.update(customer);
    String h_data = "%s  %s".formatted( warehouse.w_name.length() >
          10 ? warehouse.w_name.substring(0, 10) : warehouse.w_name,
          district.d_name.length() > 10 ?
          district.d_name.substring(0, 10) : district.d_name );
    return new PaymentOut(in.w_id, in.d_id, in.c_id, in.c_w_id,
          in.c_d_id, in.amount, h_data);
}
@Inbound(values = "order-status-in")
@Outbound("order-status-out")
@Transactional(type = R)
public OrderStatusOut processOrderStatus(OrderStatusIn in) {
    if(in.by_name){
        List<Customer> customers =
              this.customerDB.getCustomerByLastName(in.d_id,
              in.w_id, in.c_last);
    } else {
        Customer customer = this.customerDB.lookupByKey(new
              Customer.CustomerId(in.c_id, in.d_id, in.w_id));
    }
    return new OrderStatusOut(in.w_id, in.d_id, in.c_id);
}

@Inbound(values = "delivery-out")
@Transactional(type = RW)
@PartitionBy(clazz = DeliveryOut.class, method = "getId")
public void processDelivery(DeliveryOut in) {
    List<Customer.CustomerId> customerIds = new
          ArrayList<>(NUM_DIST_PER_WARE);
    for(int d_id = 1; d_id <= NUM_DIST_PER_WARE; d_id ++) {
        customerIds.add(new
              Customer.CustomerId(in.customerIds[d_id-1], d_id,
              in.w_id));
    }
    List<Customer> customers =
          this.customerDB.lookupByKeys(customerIds);
    int idx = 0;
    for(Customer customer : customers) {
        customer.c_balance += in.amounts[idx++];
        customer.c_delivery_cnt++;
    }
    this.customerDB.updateAll(customers);
}
```

Listing 3: DoMS Warehouse

```
@Repository
public interface IStockRepository extends
      IRepository<Stock.StockId, Stock> {
    @Query("select distinct s_i_id from stock where s_i_id IN
          (:itemIds) and s_w_id = :w_id and s_quantity < :threshold")
    int[] getStockCount(int[] itemIds, int w_id, int threshold);
}
@Inbound(values = "new-order-ware-out")
@Outbound("new-order-inv-out")
@Transactional(type = RW)
@PartitionBy(clazz = NewOrderWareOut.class, method = "getId")
public NewOrderInvOut processNewOrder(NewOrderWareOut in) {
    float[] prices = this.itemDB.getPricePerItemId(in.itemsIds);
    String[] ol_dist_info = new String[in.itemsIds.length];
    List<Stock> stockItemsToUpdate = new ArrayList<>(prices.length);
    for(int i = 0; i < in.itemsIds.length; i++){
        Stock stock = this.stockDB.lookupByKey(new
              Stock.StockId(in.itemsIds[i], in.supWares[i]));
        ol_dist_info[i] = stock.getDistInfo(in.d_id);
        int ol_quantity = in.qty[i];
        if(stock.s_quantity > ol_quantity){
            stock.s_quantity = stock.s_quantity - ol_quantity;
        } else {
            stock.s_quantity = stock.s_quantity - ol_quantity + 91;
        }
        stock.s_ytd = stock.s_ytd + ol_quantity;
        stock.s_order_cnt++;
        if(stock.s_w_id != in.w_id){
            stock.s_remote_cnt++;
        }
        stockItemsToUpdate.add(i, stock);
    }
    this.stockDB.updateAll(stockItemsToUpdate);
    return new NewOrderInvOut(in.w_id, in.d_id, in.c_id,
          in.itemsIds, in.supWares, in.qty, in.allLocal, in.w_tax,
          in.d_next_o_id, in.d_tax, in.c_discount, prices,
          ol_dist_info);
}
@Inbound(values = "stock-level-ord-out")
@Transactional(type = R)
public void processStockLevel(StockLevelOrdOut in) {
    int threshold = randomNumber(10, 20);
    int[] itemIds = this.stockDB.getStockCount(in.itemsIds, in.w_id,
          threshold);
}
```

Listing 4: DoMS Inventory

```
@Inbound(values = "new-order-inv-out")
@Transactional(type = W)
@Parallel
public void processNewOrder(NewOrderInvOut in){
    Order order = new Order(in.d_next_o_id, in.d_id, in.w_id,
          in.c_id, new Date(), in.itemsIds.length, in.allLocal);
    NewOrder newOrder = new NewOrder(in.d_next_o_id, in.d_id,
          in.w_id);
    this.orderDB.insert(order);
```

```
    this.newOrderDB.insert(newOrder);
    List<OrderLine> orderLinesToInsert = new
          ArrayList<>(in.itemsIds.length);
    for(int i = 0; i < in.itemsIds.length; i++){
        float ol_amount = (in.qty[i] * in.itemsIds[i] * (1 +
              in.w_tax + in.d_tax) * (1 - in.c_discount));
        OrderLine orderLine = new OrderLine(in.d_next_o_id, in.d_id,
              in.w_id,i+1, in.itemsIds[i], in.supWares[i],null,
              in.qty[i], ol_amount, in.ol_dist_info[i]);
        orderLinesToInsert.add(i, orderLine);
    }
    this.orderLineDB.insertAll(orderLinesToInsert);
}

@Inbound(values = "payment-out")
@Transactional(type = W)
@Parallel
public void processPayment(PaymentOut out){
    History history = new History(out.c_id, out.c_d_id, out.c_w_id,
          out.d_id, out.w_id, new Date(), out.amount, out.data);
    this.historyDB.insert(history);
}

@Inbound(values = "order-status-out")
@Transactional(type = R)
public void processOrderStatus(OrderStatusOut in){
    Order order = this.orderDB.getLastOrderByCustomerId(in.c_id);
    List<OrderLineInfoDto> orderLinesInfo =
          this.orderLineDB.getOrderLinesInfo(order.o_id,
          order.o_d_id, order.o_w_id);
}

@Inbound(values = "stock-level-ware-out")
@Outbound("stock-level-ord-out")
@Transactional(type = R)
public StockLevelOrdOut processStockLevel(StockLevelWareOut in) {
    int[] orderIds = IntStream.range(in.next_o_id - 20,
          in.next_o_id).toArray();
    int[] itemIds = this.orderLineDB.getAllItemsByOrderIds(orderIds,
          in.d_id, in.w_id);
    return new StockLevelOrdOut(in.w_id, itemIds);
}
```

```
@Inbound(values = "delivery-in")
@Outbound("delivery-out")
@Transactional(type = RW)
@PartitionBy(clazz = DeliveryIn.class, method = "getId")
public DeliveryOut processDelivery(DeliveryIn in) {
    int no_o_id;
    int[] customerIds = new int[10];
    float[] amounts = new float[10];
    for(int d_id = 1; d_id <= TPCcConstants.NUM_DIST_PER_WARE;
          d_id++) {

        NewOrder newOrder = newOrder =
              this.newOrderDB.getFirstNewOrder(d_id, w_id);
        this.newOrderDB.delete(newOrder);

        no_o_id = newOrder.no_o_id;
        Order order = this.orderDB.lookupByKey(new
              Order.OrderId(no_o_id, d_id, in.w_id));
        order.o_carrier_id = in.carrier_id;
        this.orderDB.update(order);

        Date date = new Date();
        List<OrderLine> orderLines =
              this.orderLineDB.getAllByOrderId(no_o_id, d_id,
              in.w_id);

        float ol_amount = 0;

        for(OrderLine orderLine : orderLines) {
            orderLine.ol_delivery_d = date;
            ol_amount += orderLine.ol_amount;
        }

        this.orderLineDB.updateAll(orderLines);

        customerIds[d_id - 1] = order.o_c_id;
        amounts[d_id - 1] = ol_amount;
    }

    return new DeliveryOut(in.w_id, customerIds, amounts);
}
```

Listing 5: DoMS Order